codegraph_safe_helper — guide

Revision: 1 **Last modified:** 2026-09-24T19:12:21Z

Source: `constitution/scripts/codegraph/codegraph_safe_helper.py` (446 lines). Citations below are `codegraph_safe_helper.py:<line>`.

Overview

Read-only, deterministic measurements that the launcher `codegraph_safe.sh` needs and that are awkward in POSIX shell (header lines 4-12; the launcher binds it as `HELPER` at `codegraph_safe.sh:112`). Seven sub-commands:

Sub-command	Purpose
`identity`	Decide from `/proc/<pid>/cmdline` whether a PID really is a CodeGraph node process (argv identity, never a substring match)
`scan`	List live CodeGraph processes that conflict with a project
`dbinfo`	JSON facts about the project's database
`pending`	Bounded count of `status='pending'` unresolved references against a threshold
`verify`	Post-run count and state checks
`scope`	Control-needled proof that no secret/excluded path is in the index
`receipt`	Check a patched runner's receipt against its patched file

It prints only paths and counts, never file contents (header lines 17-18). The database is opened `mode=ro` and nothing is written anywhere (lines 145-151; header line 46).

Prerequisites

- `python3`, standard library only: `sqlite3`, `fnmatch`, `hashlib`, `json` (header line 47).
- Linux `/proc` for `identity` and `scan` (lines 64, 92, 128).
- For the DB sub-commands: `<project>/.codegraph/codegraph.db` (line 146). The tables/columns the code reads are `files(path)`, `nodes`, `project_metadata(key, value)` and `unresolved_refs(status)` with the index `idx_unresolved_status` (lines 166-175, 190-191, 216).

Usage

```
codegraph_safe_helper.py identity <pid>
codegraph_safe_helper.py scan <project> writer|reader
codegraph_safe_helper.py dbinfo <project>
codegraph_safe_helper.py pending <project> [--threshold N]
codegraph_safe_helper.py verify <project> [--reported N] [--status-files N]
                                [--expected N --tolerance-pct P]
codegraph_safe_helper.py scope <project> <baseline-file> [<exceptions-file>]
codegraph_safe_helper.py receipt <runner-dir> <stock-version>
```

There is no `--help`: no arguments, an unknown sub-command, or wrong argument shape returns exit 2 (lines 431-437).

`identity`

`identity <pid>`: exactly one all-digit argument, PID greater than 1, else exit 2 (lines 103-108). A process is a CodeGraph process when `argv[0]`'s basename is `node` or `nodejs` AND some later argv element ends in `/lib/dist/bin/codegraph.js` (lines 80-88). A shell that merely mentions the path in its arguments is not one (docstring lines 73-75). The op is the token right after that element. The root is the process cwd for `serve` or when no positional argument exists, otherwise the normalised join of cwd and the LAST argument not starting with `-` (lines 89-100); option values are not distinguished from positionals (line 95). Output: `OK <op> <root>` (exit 0), or `not-node` / `not-codegraph` (exit 1), or `gone` (exit 2) (lines 109-114).

scan

`scan <project> writer|reader` (lines 121-142). The project is `realpath`ed. `writer` conflicts with ops `init index sync serve`; `reader` only with `init index sync` (lines 58-59, 125). The helper's own PID and PIDs <= 1 are skipped (line 132). A process is related when its (realpath) root equals the project or lies below it (lines 117-118); a process rooted in a parent of the project is not matched. One line per hit, sorted: `LIVE pid=<n> op=<op> root=<root>`; exit 1 if any, else 0.

dbinfo

`dbinfo <project>` prints one JSON object with sorted keys `accounted`, `error`, `exists`, `files`, `nodes_present`, `stamp_extraction`, `stamp_version`, `state` and exits 0 (lines 154-187, 245-249). `nodes_present` comes from a `select 1 from nodes limit 1` probe: the nodes table is never counted (lines 169-172). `state` / `accounted` / `stamp_*` are the metadata keys `index_state`, `index_files_accounted`, `indexed_with_version`, `indexed_with_extraction_version` (lines 174-183). A missing metadata table is swallowed silently (lines 184-185).

pending

`pending <project> [--threshold N]` (lines 225-242). Default threshold 150000; below 1 is exit 2. The count is bounded: `count(*)` over `select 1 from unresolved_refs indexed by idx_unresolved_status where status='pending' limit threshold+1` (lines 215-217). `INDEXED BY` turns a missing index into an error instead of a full scan; a progress handler aborts after `CG_SAFE_PENDING_PROBE_BUDGET_S` seconds (default 30; a non-numeric value falls back to 30) (lines 197-200, 210-214). Output is one line `PENDING verdict=<BELOW|AT_OR_ABOVE|UNKNOWN> counted=<n|?> threshold=<T> ...`.

verify

`verify <project> [...]` prints one `VERIFY <name> PASS|FAIL <detail>` line per check, in this order (lines 273-301):

1. `db_present` (exists and no error);

2. `index_state` (`index_state` metadata equals `complete`);
3. `files_nonzero` ;
4. `pending_zero` : bounded probe with cap 10000 (line 277); fails closed when the probe cannot run (lines 279-280);
5. `files_eq_accounted` : only when `index_files_accounted` exists; a non-integer value counts as -1 and fails (lines 284-289);
6. `files_eq_reported` (`--reported N`), `files_eq_status` (`--status-files N`);
7. `files_vs_expected` (`--expected N` , optional `--tolerance-pct P` , default 0.0): passes when `abs(files - N) <= N * P / 100` (lines 296-301).

Then `VERIFY_NOTE version_stamp PRESENT|ABSENT ...` (informational; the stamps are written only by a completed full index, and the helper never fabricates them, lines 302-310) and `VERIFY_FILES <n>` (line 311). Exit 0 only if every check passed.

`scope`

`scope <project> <baseline-file> [<exceptions-file>]` (`cmd_scope` , currently at `codegraph_safe_helper.py:404` ; line numbers elsewhere in this section predate the exceptions mechanism below and have not been re-walked — verify against the live source rather than trusting them literally, §11.4.6).

Per-path exceptions (added; `load_exceptions` / `ALLOWED_EXCEPTION_CLASSES` , currently `codegraph_safe_helper.py:341-383`). The `secret_named` (`segglob:*secret*`) and `secrets_dir` (`dir:secrets`) baseline classes are deliberately broad path-SUBSTRING heuristics — a defense-in-depth net, independent of and stricter than the real CodeGraph indexing exclude config — so on any real, large codebase they will false-positive on legitimate source that merely discusses or scans for "secrets" as a concept (a crypto HAL implementing a shared-secret protocol, a tool that scans FOR leaked credentials, a public API's own Secret-Manager service schema). Loosening the glob itself was rejected — it

would also narrow detection of a genuinely new secret-shaped file. Instead, an OPTIONAL third argument names an exceptions file: `<class> <exact-project-relative-path>` per line (`#` starts a comment), narrow and auditable —

- Only classes in `ALLOWED_EXCEPTION_CLASSES` (`secret_named` , `secrets_dir`) may appear. Any other class name (in particular the credential-CONTENT classes `env_file` / `keystore` / `signing_key` / `service_account` , §11.4.10) is a **hard error** — `SCOPE FAIL exceptions file invalid` — never silently accepted. Those four classes admit zero exceptions, by construction.
- The path MUST be exact — no `*` / `?` , no leading `/` — a pattern is a hard error too. An exception narrows exactly the ONE file it names.
- Missing/absent exceptions file ⇒ empty exceptions (fully backward-compatible with the old 2-argument call; the strictest posture, never an error).
- A listed path that doesn't actually match anything (a typo, a stale entry) changes nothing — the underlying `SCOPE_CLASS ... FAIL` persists. This is fail-closed: the file can only narrow an ALREADY-VERIFIED hit, never widen what the baseline patterns themselves catch.
- A path under a literal `secrets/` directory ALSO always matches `segglob:*secret*` (the directory-name component itself satisfies the glob), so `secrets_dir` hits are always a subset of `secret_named` hits — such a path needs an exception entry in BOTH classes to fully clear (see `scope_exceptions.txt` 's `external/googleapis/.../secretmanager_v1beta1.yaml` pair, and `tests/test_scope_exceptions.py::test_09b`).
- `SCOPE_CLASS` output gains an `excepted=<n>` field when any hits for that class were excepted, e.g. `SCOPE_CLASS secret_named kind=secret count=0 excepted=6 PASS` .

The project's own checked-in exceptions live at `constitution/scripts/codegraph/scope_exceptions.txt` , and `codegraph_safe.sh` defaults `--scope-exceptions` to that file (beside the script, mirroring how `--scope-baseline` defaults to `scope_baseline.txt`) at every `scope` call-site, including inside `supervise()` (env `CG_SV_SCOPE_EXCEPTIONS`). Tests: `tests/test_scope_exceptions.py` (13 cases) + `tests/test_scope_exceptions_mutations.sh` (7 mutants, all killed).

Baseline format (lines 316-329): one rule per line, `<class> <secret|exclude> <matcher>:<value>` ; blank lines and `#` comments are skipped; any other shape is an error. Matchers (lines 332-348), applied to project-relative paths only (an absolute path never matches):

Matcher	Matches
`exact:P`	the whole path equals P
`seg:N`	some path component equals N
`segglob:G`	some component matches glob G (`fnmatchcase`)
`dir:N`	some directory component (not the last) equals N
`topdir:N`	the first component equals N and the path has more than one component

Procedure: read `select path from files order by path` ; then a control needle (lines 375-393). The needle is the first row that is relative and exists on disk; it must be found by `exact:` , by `segglob:` on its base name and, when it has a directory, by `topdir:` , while a fabricated path (`__cg_scope_needle_absent__/<sha256[:16]>.none`) must not be found. Only then are class counts trusted. Output lines: `SCOPE_NEEDLE ... -> SEEING|BLIND` , `SCOPE_CLASS <class> kind=<kind> count=<n> PASS|FAIL` (PASS iff count 0), up to 5 `SCOPE_HIT <class> <path>` lines per failing class, then `SCOPE PASS|FAIL rows=<n> classes=<n>` .

`receipt`

`receipt <runner-dir> <stock-version>` (lines 412-427). Reads `<runner-dir>/RUNNER_RECEIPT.json` and hashes `<runner-dir>/lib/dist/db/index.js` with SHA-256. PASS requires `patched_dbjs_sha256` equal to that hash, `codegraph_version` equal to `<stock-version>` , and a non-empty `patch_id` . Output: `RECEIPT PASS|FAIL patch_id=... receipt_version=... stock_version=... sha_match=...` .

Inputs

`/proc/<pid>/cmdline` and `/proc/<pid>/cwd` (lines 64, 92); the project's `.codegraph/codegraph.db` opened read-only with a 10 s connect timeout and `busy_timeout=10000` (lines 149-150); the baseline file; the runner directory; environment variable `CG_SAFE_PENDING_PROBE_BUDGET_S` (line 210).

Outputs and exit codes

Sub-command	0	1	2	3
`identity`	codegraph process	alive but not codegraph	no such process, or usage	-
`scan`	no conflicting process	conflict	usage	-
`dbinfo`	always (given one argument)	-	usage	-
`pending`	below threshold	at or above threshold (inclusive)	usage / bad threshold	UNKNOWN (no DB, missing table or index, over budget, SQL error)
`verify`	all checks pass	any check fails	usage / bad option value	-
`scope`	PASS	FAIL, BLIND, unreadable baseline or files table	usage / unknown matcher	-
`receipt`	PASS	FAIL or unreadable	usage	-

`main()` turns an uncaught `ValueError` into `ERROR <message>` and exit 2 (lines 438-442). `verify` prints its check lines as it goes, so a bad numeric option value can produce partial output followed by `ERROR ...`.

Edge cases

- `pending <project> --threshold` with no value silently uses the default 150000, because `opt()` returns `None` when the value is missing or empty (lines 252-257, 229-231). A non-integer value (for example `3.5`) exits 2.
- `scope` counts are the SUM over a class's patterns (line 398), so a path matched by two patterns of one class is counted twice. The `kind` (`secret` / `exclude`) is printed but does not change the verdict.
- An unknown matcher name in the baseline is not rejected at load time (only the `:` is checked, lines 326-327); it raises when first used and the run ends `ERROR unknown matcher '<name>'`, exit 2.
- `scope` with no database, an empty `files` table, or no row that resolves to a file on disk reports `BLIND` (exit 1): an empty scan proves nothing (lines 368-381).
- `receipt`: a `RUNNER_RECEIPT.json` that parses but is not a JSON object is outside the `except` on line 420; it ended with an uncaught `AttributeError` traceback and exit 1 in the run below.
- `scan writer` treats a `serve` process as a conflict; `scan reader` does not (lines 58-59).

Internal behaviour

`open_ro()` returns `None` when the DB file does not exist (lines 147-148). `db_facts()` is shared by `dbinfo`, `verify`. `pending_probe()` returns `(count, None)` or `(None, reason)` and never guesses a count (lines 194-222). `related()`, `match()` and `codegraph_identity()` are pure helpers. Nothing in the file writes to disk or to a database.

Related

Guide / file	Relation
codegraph_safe	The launcher that calls every sub-command
codegraph_guard	Static gate protecting the launcher as the only writer entry
owed_report	Reports whether this tool has tests and a guide
`scope_baseline.txt`	Baseline file consumed by `scope` (header cross-refs)
`tests/test_unit_safe.sh`	Launcher unit tests (stub binaries) named in the header cross-refs

Last verified

2026-09-24T19:12:21Z — ran every sub-command from `scripts/codegraph` against a throwaway fixture in a scratch directory (a small SQLite DB with `files`, `nodes`, `project_metadata`, `unresolved_refs`; a runner directory with a receipt; a stand-in process started with `exec -a node python3 -c ... /x/lib/dist/bin/codegraph.js index <dir>`). No CodeGraph binary and no live index was touched.

- `dbinfo`: `{"accounted": "2", "error": null, "exists": true, "files": 2, "nodes_present": true, "stamp_extraction": null, "stamp_version": null, "state": "complete"}`, exit 0.
- `pending --threshold 3` with 3 pending rows: `verdict=AT_OR_ABOVE counted=3`, exit 1; `--threshold 4`: `BELOW`, exit 0; no DB: `UNKNOWN ... reason=no database`, exit 3; `--threshold 0`: exit 2; no value: default 150000, exit 0.
- `verify` with 0 pending: exit 0; with 3 pending or `--reported 5`: FAIL, exit 1; no DB: 4 FAIL lines, exit 1; `--reported abc`: `ERROR invalid literal for int()...`, exit 2.

- `scope`: `SCOPE_NEEDLE ... -> SEEING`, both classes `count=0 PASS`, exit 0; `segglob:src*` class: `count=2 FAIL` plus two `SCOPE_HIT` lines, exit 1; missing baseline / empty files table: exit 1; `weird:foo` matcher: exit 2.
- `receipt`: matching version and hash: `RECEIPT PASS`, exit 0; wrong stock version, tampered `index.js`, missing receipt: exit 1; receipt file `[]`: `AttributeError` traceback, exit 1.
- `identity` on the stand-in: `OK index <dir>`, exit 0; on the invoking shell: `not-node`, exit 1; PID 999999: `gone`, exit 2; PID 1: exit 2.
- `scan writer` and `scan reader` while the `index` stand-in ran: one `LIVE pid=... op=index root=...` line, exit 1; a different project: exit 0; after the stand-in was killed: exit 0. With a `serve` stand-in (`identity` gave `OK serve <cwd>`), `scan reader` and `scan writer` on an unrelated project both exited 0 (so this run does not show the reader-vs-serve distinction; that comes from lines 58-59 and 125).
- No arguments, unknown sub-command, `scan ... both`: exit 2.

UNCONFIRMED: `tests/test_unit_safe.sh` was not run for this guide.